

## RAPPORT DE MINI-PROJET COMPILATION

Par:

Ahmed CHAABANE

Firas KAHLAOUI

Seifeddine BEN FRADJ

# Compilateur de franC



Année Universitaire : 2025-2026

# Contents

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                 | <b>3</b>  |
| <b>2</b> | <b>Choix Technologiques</b>                                         | <b>4</b>  |
| <b>3</b> | <b>Structure de la Grammaire et de l'AST</b>                        | <b>5</b>  |
| 3.1      | L'Analyse Lexicale et Syntaxique . . . . .                          | 5         |
| 3.1.1    | Notation et Définitions (Pseudo-BNF) . . . . .                      | 5         |
| 3.1.2    | Grammaire formelle détaillée . . . . .                              | 6         |
| 3.1.3    | Décomposition des éléments clés . . . . .                           | 6         |
| 3.2      | L'Arbre Syntaxique Abstrait (AST) . . . . .                         | 7         |
| 3.3      | L'AST et le Moteur d'Exécution . . . . .                            | 8         |
| <b>4</b> | <b>Défis Rencontrés et Solutions Apportées</b>                      | <b>10</b> |
| 4.1      | Évaluation et Court-circuit des Opérateurs Logiques . . . . .       | 10        |
| 4.2      | Gestion du flux avec "retourne" . . . . .                           | 10        |
| 4.3      | Gestion du typage dynamique à l'affichage . . . . .                 | 11        |
| <b>5</b> | <b>Guide Pratique et Exemples de Code</b>                           | <b>12</b> |
| 5.1      | Variables, Types et Affichage . . . . .                             | 12        |
| 5.2      | Structures de Contrôle et Boucles . . . . .                         | 13        |
| 5.3      | Fonctions et Valeurs de Retour . . . . .                            | 13        |
| <b>6</b> | <b>Perspectives et Évolutions Futures</b>                           | <b>15</b> |
| 6.1      | Portée des Variables (Scoping) et Paramètres de Fonctions . . . . . | 15        |
| 6.2      | Structures de Données Complexes . . . . .                           | 16        |
| 6.3      | Gestion Avancée des Erreurs . . . . .                               | 16        |
| <b>7</b> | <b>Conclusion</b>                                                   | <b>17</b> |

# 1

## Introduction

Dans le cadre du module de Techniques de Compilation, ce projet vise à concevoir et implémenter un compilateur/interpréteur complet pour un langage personnalisé nommé **franC**. Ce langage se distingue par sa syntaxe francisée, rendant la programmation plus accessible tout en conservant la puissance des concepts fondamentaux de la programmation impérative. Le projet englobe l'analyse lexicale, syntaxique, sémantique, ainsi que l'exécution directe via un Arbre Syntaxique Abstrait (AST).

# 2

## Choix Technologiques

Pour garantir la robustesse et l'efficacité de l'interpréteur, les choix technologiques se sont portés sur des outils standards et éprouvés :

- **Langage d'implémentation (C)** : Le C a été choisi pour sa gestion précise de la mémoire et ses performances. Il permet de manipuler les structures complexes de l'AST et la table des symboles de manière optimale.
- **Flex (Fast Lexical Analyzer Generator)** : Utilisé pour l'analyse lexicale. Flex permet de transformer efficacement le flux de caractères source en unités lexicales (tokens) grâce aux expressions régulières.
- **Bison (GNU Parser Generator)** : Utilisé pour l'analyse syntaxique. Bison génère un analyseur LALR(1) qui valide la structure du code source selon notre grammaire formelle et construit l'AST dynamiquement.
- **Make** : Utilisé pour automatiser le processus de compilation du compilateur lui-même, en isolant proprement les fichiers générés dans un répertoire de construction (`build/`).

# 3

## Structure de la Grammaire et de l'AST

### 3.1 L'Analyse Lexicale et Syntaxique

L'analyse du code source se fait en deux étapes. L'analyseur lexical (Flex) transforme le flux de caractères en *Tokens* (Terminaux), puis l'analyseur syntaxique (Bison) valide la structure selon une grammaire formelle.

#### 3.1.1 Notation et Définitions (Pseudo-BNF)

Pour décrire la syntaxe de **franC**, nous utilisons la notation **pseudo-BNF** (Backus-Naur Form). C'est un métalangage universel permettant de définir les règles de production d'un langage.

**Légende de lecture :**

- **<Mot>** : Un **Non-Terminal**. C'est une règle complexe qui se décompose en d'autres éléments.
- **MAJUSCULE** ou **"mot"** : Un **Terminal** (Token). C'est l'élément de base final (ex: un chiffre, un mot-clé).
- **::=** : Signifie "est défini par".

- `|` : Représente le choix logique (OU).
- `[ ]` : Indique un élément **optionnel**.

### 3.1.2 Grammaire formelle détaillée

Voici la structure acceptée par l'analyseur Bison :

```

<Programme>      ::= <Liste_Instructions>
<Liste_Inst>     ::= <Instruction> | <Liste_Inst> <Instruction>
<Instruction>    ::= <Declaration> | <Affectation> | <Affichage>
                  | <Instruction_Si> | <Instruction_Boucle>
                  | <Decl_Fonction> | <Appel_Fonction> ";"
                  | "retourne" <Expression> ";"

<Declaration>    ::= ["constante"] <Type> IDENTIFIANT ["=" <Expression>] ";"
<Type>           ::= "entier" | "reel" | "caractere" | "chaine"

<Instruction_Si> ::= "si" "(" <Expression> ")" <Bloc>
                  [ "sinon" <Bloc> | "sinon" <Instruction_Si> ]

<Expression>    ::= NOMBRE | REEL | CARACTERE_VAL | CHAINE_VAL | IDENTIFIANT
                  | <Appel_Fonction>
                  | <Expression> OP_ARITHMETIQUE <Expression>
                  | <Expression> OP_LOGIQUE <Expression>
                  | "(" <Expression> ")"

```

### 3.1.3 Décomposition des éléments clés

Chaque règle de la grammaire définit la manière dont les tokens sont assemblés :

- `<Liste_Inst>` : Utilise la **récursivité gauche**. Cela permet au compilateur de traiter un nombre illimité d'instructions les unes après les autres.

- **<Declaration>** : Cette règle montre la flexibilité du langage. On peut déclarer une variable simple (`entier x;`) ou avec initialisation (`entier x = 10;`), avec ou sans le modificateur `constante`.
- **<Expression>** : C'est l'élément le plus riche. Il est composé de valeurs simples (terminaux) ou de combinaisons récursives. Par exemple, l'expression `(A + 5) > 10` est décomposée en sous-expressions imbriquées. C'est cette structure qui permet à Bison de respecter la priorité des opérations (ex: la multiplication avant l'addition).

## 3.2 L'Arbre Syntaxique Abstrait (AST)

Pendant que Bison valide la syntaxe, il construit en mémoire un **Arbre Syntaxique Abstrait**. L'AST est une représentation épurée du programme où seuls les éléments porteurs de sens (opérateurs, valeurs, noms de variables) sont conservés.

**Composition d'un nœud AST** : Chaque élément de code devient un objet `struct Noeud` contenant :

- **Le Type du Nœud** : Identifie l'action (Addition, Boucle, Affichage).
- **La Valeur** : Stocke la donnée brute (un double, une chaîne, ou le nom d'un identifiant).
- **Les Pointeurs de branchement** :
  - `gauche/droite` : Pour les opérations binaires (`3 + 4`).
  - `condition / branche_si / branche_sinon` : Pour les structures logiques.

**Moteur d'interprétation** : L'exécution se fait par un parcours récursif de l'arbre. Pour un nœud `NODE_AFFECTATION`, l'interpréteur évalue d'abord récursivement l'expression à droite du signe `=`, puis stocke le résultat final dans la Table des Symboles sous le nom contenu dans le nœud de gauche. Cette approche permet de gérer naturellement l'imbrication des fonctions et des calculs complexes.

### 3.3 L'AST et le Moteur d'Exécution

Plutôt que de générer du code machine (comme l'assembleur) ou du bytecode, **franC** utilise l'approche d'un interpréteur "Tree-Walk". Pendant que Bison valide la syntaxe, chaque règle de la grammaire alloue et relie dynamiquement une structure en mémoire pour former un Arbre Syntaxique Abstrait (AST).

L'AST capture la logique du programme en éliminant les détails syntaxiques inutiles (comme les points-virgules ou les parenthèses).

**Structure d'un Nœud** : Chaque nœud de l'arbre est défini par une structure C (`struct Noeud`) contenant :

- **type** : Une énumération indiquant la nature de l'opération (`NODE_OPERATION`, `NODE_SI`, `NODE_AFFECTATION`, etc.).
- **valeur / nom / chaine\_val** : Les données brutes stockées dans le nœud (un chiffre, le nom d'une variable, ou du texte).
- **Les pointeurs d'arborescence** : `gauche` et `droite` pour lier les enfants dans le cas d'opérations binaires (ex: `A + B`).
- **Les pointeurs de contrôle** : `condition`, `branche_si`, et `branche_sinon` utilisés spécifiquement pour les conditions et les boucles.

**Le Fonctionnement du Moteur d'Exécution** : L'exécution du programme commence une fois la racine de l'AST entièrement générée. La fonction principale `executer_ast(Noeud *n)` parcourt cet arbre de manière **récursive**.

Par exemple, pour évaluer un nœud de type `NODE_OPERATION` représentant une addition (+) :

1. Le moteur s'appelle lui-même sur le nœud `gauche` pour évaluer la première partie de l'expression.
2. Il s'appelle ensuite sur le nœud `droite` pour évaluer la seconde partie.
3. Une fois les deux valeurs obtenues (remontées par la récursivité), il effectue l'addition native en C et retourne le résultat global.



Cette récursivité permet de traiter des expressions d'une complexité infinie. Pour les structures de contrôle comme le `NODE_TANT_QUE`, l'interpréteur évalue la `condition` avec l'AST ; tant que celle-ci renvoie une valeur différente de zéro (Vrai), il appelle récursivement l'exécution sur la `branche_si` (le corps de la boucle).

# 4

## Défis Rencontrés et Solutions Apportées

### 4.1 Évaluation et Court-circuit des Opérateurs Logiques

**Défi :** L'implémentation des opérateurs ET et OU nécessitait une évaluation conditionnelle pour éviter des erreurs ou des calculs inutiles (court-circuit).

**Solution :** Modification de la fonction d'exécution de l'AST pour évaluer d'abord la branche gauche. Si le résultat de la branche gauche détermine l'issue finale (ex: gauche = faux pour un ET), la branche droite n'est tout simplement pas parcourue.

### 4.2 Gestion du flux avec "retourne"

**Défi :** Dans un bloc de code (séquence), l'apparition du mot-clé `retourne` doit arrêter immédiatement l'exécution des instructions suivantes et propager la valeur à l'appelant.

**Solution :** Introduction de variables globales d'état (`retour_declenche` et `valeur_retour`). Lors de l'exécution d'un `NODE_SEQUENCE`, le moteur vérifie si le drapeau de retour est levé avant de procéder à l'instruction suivante. Le contexte du drapeau est sauvegardé lors des appels imbriqués pour garantir un comportement correct lors de la récursion.

## 4.3 Gestion du typage dynamique à l’affichage

**Défi :** Imprimer correctement le contenu d’une variable sans forcer l’utilisateur à spécifier le type (comme en C avec `%d` ou `%s`).

**Solution :** Le nœud `NODE_AFFICHER` a été enrichi pour interroger la table des symboles dynamiquement. Si la variable est de type `VAR_CHAINE`, il utilise `%s` ; si elle est `VAR_CARACTERE`, il utilise `%c` ; sinon, il l’évalue comme un nombre avec `%g`.

# 5

## Guide Pratique et Exemples de Code

Afin d'illustrer concrètement la syntaxe et les capacités du langage **franC**, ce chapitre présente plusieurs exemples d'utilisation, allant de la déclaration de variables à l'implémentation d'une logique algorithmique complète.

### 5.1 Variables, Types et Affichage

Le langage supporte quatre types fondamentaux ainsi que la notion de constante. La fonction native `afficher()` adapte dynamiquement sa sortie en fonction du type de la variable qui lui est passée.

```
// Déclaration de variables simples
entier age = 22;
reel pi_approx = 3.1415;
caractere initiale = 'F';
chaine salutation = "Bonjour tout le monde !";

// Déclaration d'une constante (immuable)
constante entier MAX_UTILISATEURS = 100;
```

```
// Affichage dynamique (le moteur sait comment formater chaque type)
afficher(salutation);
afficher("Age de l'utilisateur :");
afficher(age);
```

## 5.2 Structures de Contrôle et Boucles

**franC** permet de gérer le flux d'exécution grâce aux embranchements conditionnels et aux boucles `tant_que`. Les opérateurs logiques `et` et `ou` permettent de créer des conditions complexes avec évaluation par court-circuit.

```
entier compteur = 5;

// Boucle conditionnelle
tant_que (compteur > 0) {
    si (compteur == 3) {
        afficher("Attention : le compteur est exactement à 3 !");
    } sinon si (compteur > 3) {
        afficher("Le compteur est élevé.");
    } sinon {
        afficher("Le compteur est bas.");
    }

    // Décrémentement
    compteur = compteur - 1;
}
```

## 5.3 Fonctions et Valeurs de Retour

La déclaration de sous-programmes se fait avec le mot-clé `fonction`. L'instruction `retourne` permet d'interrompre l'exécution du bloc courant et de renvoyer le résultat à l'instruction appelante.

```
entier base = 10;
entier multiplicateur = 5;

// Définition d'une fonction (utilisant les variables globales)
fonction calculer_produit() {
    si (base <= 0 ou multiplicateur <= 0) {
        retourne 0; // Court-circuit si les valeurs sont invalides
    }
    retourne base * multiplicateur;
}

// Appel de la fonction, affectation et affichage
entier resultat = calculer_produit();

afficher("Le résultat du produit est :");
afficher(resultat);
```

# 6

## Perspectives et Évolutions Futures

Bien que le langage **franC** réponde aux exigences initiales du cahier des charges et propose un environnement d'exécution fonctionnel, ce projet pose les fondations d'un interpréteur qui pourrait être grandement enrichi. Voici les principales pistes d'amélioration envisagées pour les prochaines itérations :

### 6.1 Portée des Variables (Scoping) et Paramètres de Fonctions

Actuellement, la Table des Symboles gère toutes les variables de manière globale. Une évolution majeure consisterait à implémenter des environnements locaux (une pile de tables de symboles). Cela permettrait de :

- Créer des variables locales propres à chaque bloc `{}` ou fonction.
- Permettre le passage d'arguments aux fonctions (ex: `fonction addition(entier a, entier b)`), rendant le code beaucoup plus modulaire et évitant l'utilisation systématique de variables globales.

## 6.2 Structures de Données Complexes

Pour étendre les capacités algorithmiques du langage, l'ajout de structures de données agrégées est indispensable :

- **Les Tableaux** : Permettre la déclaration et l'itération sur des listes d'éléments (ex: `entier tab[10];`).
- **Types personnalisés (Structures)** : Permettre à l'utilisateur de regrouper plusieurs variables sous une même entité logique (similaire au `struct` en C).

## 6.3 Gestion Avancée des Erreurs

Bien que l'analyseur signale les erreurs de syntaxe, le système pourrait être amélioré par des mécanismes de récupération d'erreurs (Error Recovery). Plutôt que de s'arrêter à la première anomalie, le compilateur pourrait identifier l'erreur, la signaler précisément avec la ligne et la colonne, puis continuer l'analyse pour remonter toutes les erreurs du fichier source en une seule exécution.



# 7

## Conclusion

Le développement du langage **franC** de bout en bout a représenté un défi technique particulièrement enrichissant. Il a permis de concrétiser les concepts théoriques exigeants abordés en cours — tels que les expressions régulières, les grammaires hors contexte et la théorie des analyseurs LALR(1) — en un véritable produit logiciel.

Ce projet a abouti à la création d'un interpréteur modulaire doté de fonctionnalités avancées. La gestion d'un typage riche, le traitement des constantes, l'évaluation par court-circuit et le support des fonctions avec valeur de retour prouvent la maturité de l'architecture choisie. Surtout, la maîtrise d'outils standards de l'industrie comme Flex, Bison et l'automatisation via Make a permis de bâtir un environnement de développement sain et évolutif en langage C.

En définitive, concevoir ce compilateur nous a offert une compréhension profonde et transparente de la mécanique interne des langages de programmation que nous utilisons au quotidien. Si **franC** peut encore évoluer pour intégrer des structures de données complexes ou une véritable machine virtuelle, les fondations posées par son moteur d'évaluation par arbre syntaxique sont d'ores et déjà un franc succès.